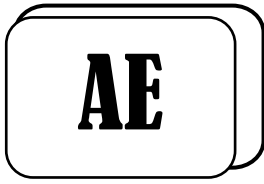




POLITEKNIK MANUFAKTUR BANDUNG

Jalan Kanayakan 21 Tlp/Fax. +62 (022) 7312564 Bandung 40275
Telp. (022)2500241 Faks. (022)2502649 <https://polman-bandung.ac.id/>



LAPORAN TUGAS MANDIRI 1 PEMROGRAMAN BERBASIS OBJEK (Minggu ke-1)



Nama : Syaroful Haidar Sudaryono
NIM : 225443051
Kelas : 2AEC2

**DAFTAR ISI**

DAFTAR ISI.....	1
PENDAHULUAN.....	2
1.1. Latar Belakang dan Tujuan	2
1.2. Ruang Lingkup Sistem.....	2
PRAKTIKUM HARI KE - 1.....	4
2. Desain Sistem.....	4
2.1. Class Diagram.....	4
3. Implementasi.....	5
3.1. Penjelasan Kelas dan Metode.....	5
4. Demonstrasi Program	6
4.1. Inisialisasi Sistem dan Tambah Item.....	6
4.2. Skenario Peminjaman (Skenario A).....	6
4.3. Skenario Pengembalian & Perubahan Status (Skenario B)	7
4.4. Demonstrasi Fitur OOP Kotlin.....	7
4.5. Laporan Akhir Perpustakaan	8
5. Analisis OOP.....	9
5.1. Encapsulation.....	9
5.2. Pewarisan (Inheritance)	9
5.3. Polimorfisme	9
5.4. Sealed Class	10
KESIMPULAN.....	11
6. LAMPIRAN.....	12
LAMPIRAN A: Source Code Latihan Pet Shop.....	12
LAMPIRAN B: Source Code Tugas Perpustakaan Digital.....	18



PENDAHULUAN

1.1. Latar Belakang dan Tujuan

Laporan ini disusun untuk memenuhi evaluasi Tugas Mandiri Pemrograman Berorientasi Objek. Tujuan utama pengerjaan tugas ini adalah untuk menguji kemampuan dalam menerjemahkan spesifikasi kebutuhan ke dalam struktur kelas OOP, serta mengimplementasikan 4 Pilar OOP (Enkapsulasi, Pewarisan, Polimorfisme, Abstraksi) menggunakan bahasa Kotlin. Selain itu, proyek ini difokuskan pada pemanfaatan fitur modern Kotlin seperti *Sealed Class*, *Smart Casting*, dan *Null Safety* untuk menciptakan kode yang bersih dan terstruktur.

1.2. Ruang Lingkup Sistem

Sistem Manajemen Perpustakaan Digital yang dibangun dirancang untuk dapat mengelola berbagai entitas utama, meliputi Item perpustakaan (Buku, Jurnal, DVD), Anggota perpustakaan, pencatatan Transaksi peminjaman dan pengembalian, pemantauan Status transaksi, hingga kalkulasi Denda atas keterlambatan.

(Catatan Pengerjaan: Sebelum membangun sistem utama perpustakaan ini, saya juga telah menyelesaikan latihan terbimbing pembuatan sistem Pet Shop dari Level 1 hingga 7 sebagai fondasi pemahaman sintaks.)



```
=== 1. TAMBAH HEWAN ===
```

- ✓ Milo ditambahkan ke MeowWoof Pet Shop.
- ✓ Rex ditambahkan ke MeowWoof Pet Shop.
- ✓ Chirpy ditambahkan ke MeowWoof Pet Shop.

```
=== 2. DAFTAR PEMILIK ===
```

- ✓ Pemilik Syaroful Haidar Sudaryono berhasil didaftarkan.

```
=== DAFTAR HEWAN ===
```

```
[H001] Milo | Kucing | 2 th | Tersedia
```

```
    Biaya/hari: Rp 50000.0
```

```
    Warna bulu: Oren
```

```
[H003] Rex | Anjing | 4 th | Tersedia
```

```
    Biaya/hari: Rp 80000.0
```

```
    Ras: German Shepherd
```

```
[H005] Chirpy | Burung | 1 th | Tersedia
```

```
    Biaya/hari: Rp 30000.0
```

```
    bisa terbang
```

```
=== 4. ADOPSI ===
```

- ✓ Milo berhasil diajukan untuk diadopsi.
- ✓ Syaroful Haidar Sudaryono berhasil mengajukan adopsi Milo.
- ✓ Rex berhasil diajukan untuk diadopsi.
- ✓ Syaroful Haidar Sudaryono berhasil mengajukan adopsi Rex.
- ✗ Syaroful Haidar Sudaryono sudah punya 2 adopsi aktif. Selesaikan dulu.

```
=== HEWAN TERSEDIA ===
```

```
[petshop.Burung@87aac27.id] Chirpy (Burung)
```

```
=== LAPORAN PET SHOP ===
```

```
Total hewan      : 3
```

```
Hewan tersedia: 1
```

```
Total adopsi    : 2
```

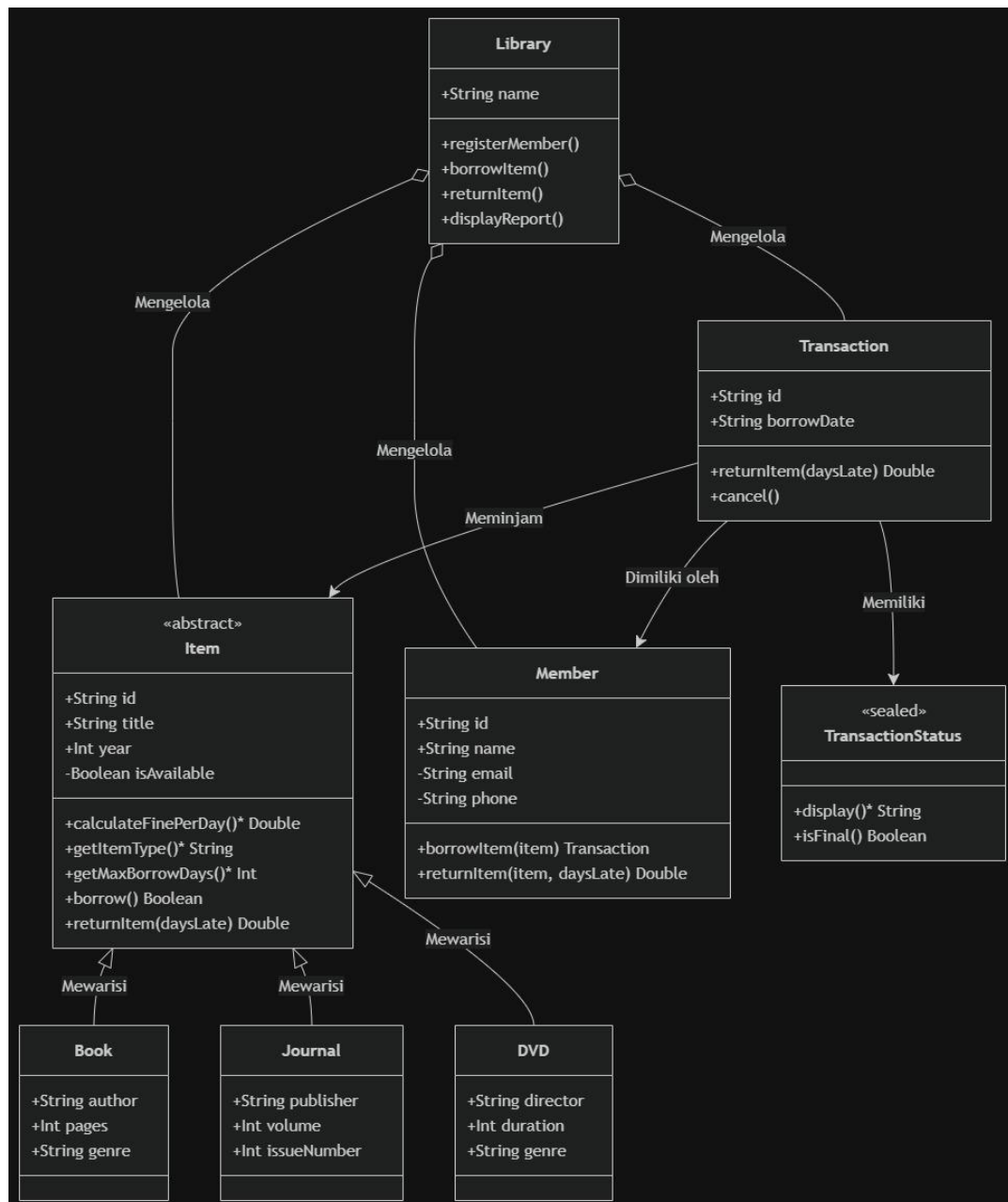


PRAKTIKUM HARI KE - 1

2. Desain Sistem

2.1. Class Diagram

Sistem ini berpusat pada kelas abstrak Item yang menjadi induk, kemudian menurunkan sifatnya ke sub-kelas Book, Journal, dan DVD. Kelas Member saling terhubung dengan Item melalui kelas Transaction yang bertindak sebagai pencatat riwayat peminjaman, di mana setiap transaksi dibatasi oleh aturan mutlak dari *sealed class* TransactionStatus. Seluruh interaksi kelas ini dibungkus dan dikelola secara terpusat oleh kelas Library.





3. Implementasi

3.1. Penjelasan Kelas dan Metode

- **Kelas Item & Sub-kelas:** `Item` merepresentasikan item umum yang bersifat abstrak, sehingga tidak bisa diinstansiasi secara langsung. Detail perilakunya direalisasikan pada `Book`, `Journal`, dan `DVD` yang meng-*override* metode kalkulasi denda sesuai spesifikasinya.
- **Kelas TransactionStatus:** Digunakan untuk membatasi status transaksi peminjaman agar kondisinya diketahui pada masa *compile-time*.
- **Kelas Transaction & Member:** `Transaction` merepresentasikan satu transaksi peminjaman yang menghubungkan item, anggota, dan status. `Member` memuat data privasi anggota perpustakaan yang dapat meminjam item.
- **Kelas Library:** Berfungsi sebagai kelas utama yang menyimpan keseluruhan *list* item, anggota, dan transaksi dalam bentuk koleksi data `MutableList`.



4. Demonstrasi Program

Berikut adalah hasil eksekusi program Main.kt yang menjalankan 14 skenario urutan wajib:

4.1. Inisialisasi Sistem dan Tambah Item

Sistem berhasil mendaftarkan 2 Buku, 2 Jurnal, dan 2 DVD beserta 3 Anggota baru.

```
=== INISIALISASI Perpustakaan Kampus ===  
Item B001 ditambahkan.  
Item B002 ditambahkan.  
Item J001 ditambahkan.  
Item J002 ditambahkan.  
Item D001 ditambahkan.  
Item D002 ditambahkan.  
Anggota Ahmad Fauzi berhasil didaftarkan.  
Anggota Dewi Lestari berhasil didaftarkan.  
Anggota Rizky Pratama berhasil didaftarkan.
```

```
=== SEMUA ITEM (6, tersedia 6) ===  
ID: B001 | Judul: Pemrograman Kotlin | Tahun: 2023  
Jenis: Buku | Status: Tersedia  
Denda/hari: Rp 2.000 | Maks. pinjam: 14 hari  
Penulis: Budi Santoso | Halaman: 350 | Genre: Programming  
  
ID: B002 | Judul: Dasar-Dasar OOP | Tahun: 2022  
Jenis: Buku | Status: Tersedia  
Denda/hari: Rp 2.000 | Maks. pinjam: 14 hari  
Penulis: Siti Rahayu | Halaman: 280 | Genre: Education  
  
ID: J001 | Judul: Jurnal Teknologi Informasi | Tahun: 2023  
Jenis: Jurnal | Status: Tersedia  
Denda/hari: Rp 3.000 | Maks. pinjam: 7 hari  
Penerbit: ITB | Volume: 15 | Edisi: 2  
  
ID: J002 | Judul: Jurnal Pendidikan | Tahun: 2022  
Jenis: Jurnal | Status: Tersedia  
Denda/hari: Rp 3.000 | Maks. pinjam: 7 hari  
Penerbit: UGM | Volume: 10 | Edisi: 1  
  
ID: D001 | Judul: Inception | Tahun: 2010  
Jenis: DVD | Status: Tersedia  
Denda/hari: Rp 5.000 | Maks. pinjam: 3 hari  
Sutradara: Christopher Nolan | Durasi: 148 menit | Genre: Sci-Fi  
  
ID: D002 | Judul: The Matrix | Tahun: 1999  
Jenis: DVD | Status: Tersedia  
Denda/hari: Rp 5.000 | Maks. pinjam: 3 hari
```

4.2. Skenario Peminjaman (Skenario A)

Aksi peminjaman oleh Ahmad, Dewi, dan Rizky terekam dan secara otomatis memotong ketersediaan item di perpustakaan.



```
=== PEMINJAMAN ===
```

```
Item 'Pemrograman Kotlin' berhasil dipinjam.  
Ahmad Fauzi berhasil meminjam 'Pemrograman Kotlin'.  
Item 'Inception' berhasil dipinjam.  
Ahmad Fauzi berhasil meminjam 'Inception'.  
Item 'Jurnal Teknologi Informasi' berhasil dipinjam.  
Dewi Lestari berhasil meminjam 'Jurnal Teknologi Informasi'.  
Item 'Dasar-Dasar OOP' berhasil dipinjam.  
Rizky Pratama berhasil meminjam 'Dasar-Dasar OOP'.
```

```
=== ITEM TERSEDIA ===
```

```
J002 - Jurnal Pendidikan (Jurnal)  
D002 - The Matrix (DVD)  
Riwayat transaksi Ahmad Fauzi:  
TRX-1789387735056 | Pemrograman Kotlin (B001) | Ahmad Fauzi | 2026-09-14 | Dipinjam  
TRX-1789387735068 | Inception (D001) | Ahmad Fauzi | 2026-09-14 | Dipinjam  
Riwayat transaksi Dewi Lestari:  
TRX-1789387735068 | Jurnal Teknologi Informasi (J001) | Dewi Lestari | 2026-09-14 | Dipinjam
```

4.3. Skenario Pengembalian & Perubahan Status (Skenario B)

Sistem berhasil mengalkulasi pengembalian tepat waktu (denda Rp 0) dan keterlambatan 3 hari pada jurnal.

```
Riwayat transaksi Ahmad Fauzi:  
TRX-1789387735056 | Pemrograman Kotlin (B001) | Ahmad Fauzi | 2026-09-14 | Dipinjam  
TRX-1789387735068 | Inception (D001) | Ahmad Fauzi | 2026-09-14 | Dipinjam  
Riwayat transaksi Dewi Lestari:  
TRX-1789387735068 | Jurnal Teknologi Informasi (J001) | Dewi Lestari | 2026-09-14 | Dipinjam
```

```
=== PENGEMBALIAN ===
```

```
Item 'Pemrograman Kotlin' berhasil dikembalikan.  
Item 'Jurnal Teknologi Informasi' berhasil dikembalikan.  
Total denda: Rp 9.000
```

```
Riwayat transaksi Ahmad Fauzi:  
TRX-1789387735056 | Pemrograman Kotlin (B001) | Ahmad Fauzi | 2026-09-14 | Dikembalikan  
TRX-1789387735068 | Inception (D001) | Ahmad Fauzi | 2026-09-14 | Dipinjam  
Riwayat transaksi Dewi Lestari:  
TRX-1789387735068 | Jurnal Teknologi Informasi (J001) | Dewi Lestari | 2026-09-14 | Terlambat (3 hari)
```

4.4. Demonstrasi Fitur OOP Kotlin

Pengujian Polimorfisme, *Sealed Class*, *Smart Casting*, dan Enkapsulasi berjalan lancar sesuai pembatasan akses data.



```
=== POLIMORFISME ===
```

```
Buku - Denda/hari: Rp 2.000
```

```
Jurnal - Denda/hari: Rp 3.000
```

```
DVD - Denda/hari: Rp 5.000
```

```
=== SEALED CLASS ===
```

```
Dipinjam
```

```
Dikembalikan
```

```
Terlambat (5 hari)
```

```
Dibatalkan
```

```
=== SMART CASTING ===
```

```
Item B001 adalah Buku
```

```
Casting B001 ke DVD: null (bukan DVD)
```

```
=== ENKAPSULASI ===
```

```
'isAvailable' tidak dapat diubah langsung karena private set. Email anggota juga private dan diakses melalui getEmail().
```

4.5. Laporan Akhir Perpustakaan

```
=== LAPORAN Perpustakaan Kampus ===
```

```
Total item: 6
```

```
Tersedia: 4
```

```
Dipinjam: 2
```

```
Total anggota: 3
```

```
Total transaksi: 4
```

```
Total denda: Rp 9.000
```

```
Process finished with exit code 0
```



5. Analisis OOP

5.1. Encapsulation

1. Contoh Penerapan:

- Properti `isAvailable` di kelas `Item` menggunakan *visibility modifier* `private set`.
- Properti `email` di kelas `Member` menggunakan `private val`.
- Properti `phone` di kelas `Member` menggunakan `private val`.

2. **Alasan `private set` pada `isAvailable`:** Agar status ketersediaan hanya bisa diubah dari dalam logika internal kelas (melalui metode `borrow()` dan `returnItem()`). Jika dibuat `public var`, entitas luar bisa memanipulasi ketersediaan item sesuka hati tanpa melalui prosedur transaksi yang sah.

3. **Alasan `private val` pada `email` dan `phone`:** Bertujuan menyembunyikan data privasi dari akses langsung. Data ini hanya dapat dibaca melalui *getter* publik resmi seperti `getEmail()` dan `getPhone()`.

5.2. Pewarisan (Inheritance)

1. **Diagram Hierarki:** Kelas Induk adalah `Item`. Sub-kelas yang menjadi turunannya adalah `Book`, `Journal`, dan `DVD`.

2. Penggunaan Keyword:

- `open`: Diterapkan pada fun `displayInfo()` di kelas `Item` agar subclass diizinkan untuk menambahkan perilakunya sendiri.
- `override`: Diterapkan pada fungsi `calculateFinePerDay()` di subclass untuk menimpa aturan nilai denda dari kelas induk.
- `super`: Digunakan dalam `super.displayInfo()` pada subclass untuk tetap memanggil logika dasar pencetakan informasi dari kelas induk.

3. **Alasan `Item` menjadi `abstract class`:** Konsep "`Item`" secara umum tidak memiliki wujud fisik yang nyata untuk diciptakan; wujud aslinya baru ada ketika menjadi `Buku`, `Jurnal`, atau `DVD`. Keuntungannya, kita bisa memaksa (*force*) setiap turunannya untuk wajib mendefinisikan metode `calculateFinePerDay()` dan `getMaxBorrowDays()` miliknya sendiri.

5.3. Polimorfisme

1. **Penerapan Polymorphic References:** Terjadi ketika koleksi `List<Item>` diisi oleh objek dari sub-kelas (contoh: `val polyItems: List<Item> = listOf(b2, j2, d2)`). Variabel bertipe `Item` mampu menampung objek `Book`, `Journal`, maupun `DVD`.

2. **Kerja `calculateFinePerDay()` secara Polimorfik:** Saat metode tersebut dipanggil di dalam perulangan `polyItems.forEach`, program secara otomatis mengenali tipe asli dari objek yang sedang diproses. Pemanggilan yang sama menghasilkan 2000.0 jika objeknya `Buku` dan 5000.0 jika objeknya `DVD`.

3. Penggunaan Smart Casting:

- **Keyword `is`:** Mengecek tipe data sekaligus mengonversinya secara aman. Contoh: `if (testItem is Book) { println(testItem.genre) }`.



- **Keyword as?:** Mencoba melakukan konversi (Safe Cast), dan akan mengembalikan nilai null apabila gagal tanpa membuat aplikasi *crash*. Contoh: `val castToDvd = testItem as? DVD`.

5.4. Sealed Class

1. **Mengapa sealed class:** Karena status transaksi hanya memiliki beberapa kemungkinan pasti yang mutlak. *Sealed class* memastikan semua kemungkinan turunan (Borrowed, Returned, Overdue, Cancelled) diketahui secara terpusat pada *compile-time*.
2. **Keuntungan pada when Expression:** Penggunaan *sealed class* menghilangkan keharusan pengembang untuk menuliskan blok `else` pada pengecekan `when`, sehingga kode lebih aman dari status yang tidak terpetakan.
3. **Ekshaustif when:** Artinya kompiler Kotlin menyadari bahwa seluruh skenario status telah tertangani. Contoh dari kode saya:

```
when (status) {  
    is TransactionStatus.Borrowed -> status.display()  
    is TransactionStatus.Returned -> status.display()  
    is TransactionStatus.Overdue -> status.display()  
    is TransactionStatus.Cancelled -> status.display()  
    // Tidak perlu "else" karena sudah ekshaustif  
}
```



KESIMPULAN

Penerapan Pemrograman Berorientasi Objek dengan Kotlin sangat efektif dalam merancang sistem berlapis yang aman. Empat pilar OOP (Enkapsulasi, Pewarisan, Polimorfisme, dan Abstraksi) yang diterapkan membuat pengelolaan logika Perpustakaan Digital menjadi sangat terstruktur. Modifikasi yang tidak sah berhasil dicegah menggunakan pembatasan akses data, sedangkan penulisan kondisi bersyarat menjadi lebih tangguh berkat fitur bawaan *Null Safety* dan evaluasi ekshaustif dari *Sealed Class*.



6. LAMPIRAN

LAMPIRAN A: Source Code Latihan Pet Shop

```
// 1. Hewan.kt
package petshop

abstract class Hewan(val id: String, val nama: String, val umur: Int) {
    var tersedia: Boolean = true
    private set

    abstract fun biayaPerawatanHarian(): Double
    abstract fun jenisHewan(): String

    fun ajukanAdopsi(): Boolean {
        return if (tersedia) {
            tersedia = false
            println("☑ $nama berhasil diajukan untuk diadopsi.")
            true
        } else {
            println("✗ $nama tidak tersedia.")
            false
        }
    }
}

// Pengerjaan Latihan Mandiri: Fitur Batalkan Adopsi
fun batalkanAdopsi() {
    if (!tersedia) {
        tersedia = true
        println("🔄 Adopsi $nama dibatalkan, hewan kembali tersedia.")
    }
}

open fun displayInfo() {
    val status = if (tersedia) "Tersedia" else "Tidak tersedia"
    println("[\$id] $nama | ${jenisHewan()} | $umur th | $status")
    println("    Biaya/hari: Rp ${biayaPerawatanHarian()}")
}

```[cite: 15]

2. Kucing.kt
```kotlin
// 2. Kucing.kt
package petshop

class Kucing(id: String, nama: String, umur: Int, val warnaBulu: String) :
    Hewan(id, nama, umur) {
    override fun biayaPerawatanHarian() = 50_000.0
}
```



```
        override fun jenisHewan() = "Kucing"
        override fun displayInfo() {
            super.displayInfo()
            println("    Warna bulu: $warnaBulu")
        }
    }
}
```[cite: 16]
```

```

3. Anjing.kt
```kotlin
// 3. Anjing.kt
package petshop
```

```
class Anjing(id: String, nama: String, umur: Int, val ras: String) : Hewan(id,
nama, umur) {
    override fun biayaPerawatanHarian() = 80_000.0
    override fun jenisHewan() = "Anjing"
    override fun displayInfo() {
        super.displayInfo()
        println("    Ras: $ras")
    }
}
```[cite: 12]
```

```

4. Burung.kt
```kotlin
// 4. Burung.kt (Pengerjaan Latihan Mandiri: Subclass Baru)
package petshop
```

```
class Burung(id: String, nama: String, umur: Int, val bisaTerbang: Boolean) :
Hewan(id, nama, umur) {
    override fun biayaPerawatanHarian() = 30_000.0
    override fun jenisHewan() = "Burung"
    override fun displayInfo() {
        super.displayInfo()
        val tb = if (bisaTerbang) "bisa" else "tidak bisa"
        println("    $tb terbang")
    }
}
```[cite: 13]
```

```

5. StatusAdopsi.kt
```kotlin
// 5. StatusAdopsi.kt
package petshop
```

```
sealed class StatusAdopsi {
    abstract fun deskripsi(): String
```



```
open fun isFinal(): Boolean = false
```

```
object Tersedia : StatusAdopsi() { override fun deskripsi() = "🟡 Tersedia"
}
data class Diajukan(val namaCalon: String) : StatusAdopsi() { override fun
deskripsi() = "🟡 Diajukan oleh $namaCalon" }
data class SudahDiadopsi(val tanggal: String) : StatusAdopsi() {
    override fun deskripsi() = "✅ Diadopsi pada $tanggal"
    override fun isFinal() = true
}
object Dibatalkan : StatusAdopsi() {
    override fun deskripsi() = "❌ Dibatalkan"
    override fun isFinal() = true
}
}
```[cite: 19]
```

```
6. Adopsi.kt
```

```
```kotlin
```

```
// 6. Adopsi.kt
```

```
package petshop
```

```
import java.time.LocalDate
```

```
class Adopsi(val id: String, val hewan: Hewan, val pemilik: CalonPemilik, val
tanggal: String = LocalDate.now().toString()) {
    var status: StatusAdopsi = StatusAdopsi.Tersedia
```

```
    fun finalisasi(): Boolean {
        if (status.isFinal()) {
            println("❌ Adopsi #$id sudah final, tidak bisa difinalisasi
lagi.")
            return false
        }
        status = StatusAdopsi.SudahDiadopsi(tanggal)
        println("🐾 ${hewan.nama} resmi diadopsi oleh ${pemilik.nama}!")
        return true
    }
}
```

```
// Pengerjaan Latihan Mandiri: Fitur Batalkan Adopsi
```

```
fun batalkan() {
    if (status.isFinal()) {
        println("❌ Adopsi #$id sudah final, tidak bisa dibatalkan.")
        return
    }
    status = StatusAdopsi.Dibatalkan
    hewan.batalkanAdopsi()
}
```



```
fun display() {  
    println("Adopsi #${id} | Hewan: ${hewan.nama} | Pemilik: ${pemilik.nama}  
    | Status: ${status.deskripsi()}")  
}  
}  
```[cite: 11]  

7. CalonPemilik.kt
```kotlin  
// 7. CalonPemilik.kt  
package petshop  
  
class CalonPemilik(val id: String, val nama: String, private val email: String,  
private val telepon: String) {  
    private val riwayatAdopsi: MutableList<Adopsi> = mutableListOf()  
    val jumlahAdopsi: Int get() = riwayatAdopsi.size  
    val adopsiAktif: Int get() = riwayatAdopsi.count { !it.status.isFinal() }  
  
    fun ajukanAdopsi(hewan: Hewan): Adopsi? {  
        if (!hewan.tersedia) {  
            println("✗ ${hewan.nama} sedang tidak tersedia.")  
            return null  
        }  
        // Pengerjaan Latihan Mandiri: Batas 2 Adopsi Aktif  
        if (adopsiAktif >= 2) {  
            println("✗ $nama sudah punya 2 adopsi aktif. Selesaikan dulu.")  
            return null  
        }  
        if (!hewan.ajukanAdopsi()) return null  
  
        val adopsi = Adopsi("ADP-${System.currentTimeMillis()}", hewan, this)  
        riwayatAdopsi.add(adopsi)  
        println("☑ $nama berhasil mengajukan adopsi ${hewan.nama}.")  
        return adopsi  
    }  
  
    fun displayInfo() {  
        println("[${id}] $nama | Aktif: $adopsiAktif")  
    }  
}  
```[cite: 14]  

8. PetShop.kt
```kotlin  
// 8. PetShop.kt  
package petshop
```



```
class PetShop(val nama: String) {
    private val hewanList: MutableList<Hewan> = mutableListOf()
    private val pemilikList: MutableList<CalonPemilik> = mutableListOf()
    private val adopsiList: MutableList<Adopsi> = mutableListOf()

    val totalHewan: Int get() = hewanList.size
    val hewanTersedia: Int get() = hewanList.count { it.tersedia }
    val totalPemilik: Int get() = pemilikList.size
    val totalAdopsi: Int get() = adopsiList.size

    fun tambahHewan(vararg list: Hewan) {
        list.forEach {
            hewanList.add(it)
            println("☑️ ${it.nama} ditambahkan ke $nama.")
        }
    }

    fun daftarPemilik(id: String, nama: String, email: String, telp: String): Boolean {
        if (pemilikList.any { it.id == id }) return false
        pemilikList.add(CalonPemilik(id, nama, email, telp))
        println("☑️ Pemilik $nama berhasil didaftarkan.")
        return true
    }

    fun ajukanAdopsi(pemilikId: String, hewanId: String): Adopsi? {
        val p = pemilikList.find { it.id == pemilikId } ?: return null
        val h = hewanList.find { it.id == hewanId } ?: return null

        val adopsi = p.ajukanAdopsi(h) ?: return null
        adopsiList.add(adopsi)
        return adopsi
    }

    fun displaySemuaHewan() {
        println("\n=== DAFTAR HEWAN ===")
        hewanList.forEach { it.displayInfo() }
    }

    fun displayHewanTersedia() {
        println("\n=== HEWAN TERSEDIA ===")
        val tersedia = hewanList.filter { it.tersedia }
        if (tersedia.isEmpty()) println("(tidak ada)")
        else tersedia.forEach { println("[$it.id] ${it.nama} (${it.jenisHewan()})") }
    }

    fun displayLaporan() {
```



```
println("\n=== LAPORAN PET SHOP ===")
println("Total hewan : $totalHewan")
println("Hewan tersedia: $hewanTersedia")
println("Total adopsi : $totalAdopsi")
}
}
```[cite: 18]

9. Main.kt (Pet Shop)
```kotlin
// 9. Main.kt
package petshop

fun main() {
    val shop = PetShop("MeowWoof Pet Shop")

    println("\n=== 1. TAMBAH HEWAN ===")
    shop.tambahHewan(
        Kucing("H001", "Milo", 2, "Oren"),
        Anjing("H003", "Rex", 4, "German Shepherd"),
        Burung("H005", "Chirpy", 1, true) // Implementasi Latihan Mandiri
    )

    println("\n=== 2. DAFTAR PEMILIK ===")
    shop.daftarPemilik("P001", "Syaroful Haidar Sudaryono",
        "syaroful@mail.com", "081111")
    shop.displaySemuaHewan()

    println("\n=== 4. ADOPSI ===")
    shop.ajukanAdopsi("P001", "H001")

    // Simulasi penolakan batas adopsi (Latihan Mandiri)
    shop.ajukanAdopsi("P001", "H003")
    shop.ajukanAdopsi("P001", "H005") // Akan ditolak karena sudah 2

    shop.displayHewanTersedia()
    shop.displayLaporan()
}
```[cite: 17]

```



## LAMPIRAN B: Source Code Tugas Perpustakaan Digital

```

1. Item.kt
`kotlin
/** Kelas dasar untuk semua item yang dapat dipinjam dari perpustakaan. */
abstract class Item(
 /** Pengidentifikasi item yang unik. */ val id: String,
 /** Judul item. */ val title: String,
 /** Tahun terbit atau rilis. */ val year: Int
) {
 /** Menunjukkan apakah item saat ini dapat dipinjam. */
 var isAvailable: Boolean = true
 private set

 /** Menghitung denda untuk satu hari keterlambatan. */
 abstract fun calculateFinePerDay(): Double
 /** Mengembalikan kategori item. */
 abstract fun getItemType(): String
 /** Mengembalikan batas maksimum lama peminjaman dalam hari. */
 abstract fun getMaxBorrowDays(): Int

 /** Meminjam item ini jika tersedia. */
 fun borrow(): Boolean = if (isAvailable) {
 isAvailable = false
 println("Item '$title' berhasil dipinjam.")
 true
 } else {
 println("Item '$title' sedang tidak tersedia.")
 false
 }

 /** Mengembalikan item ini dan menghitung denda keterlambatannya. */
 fun returnItem(daysLate: Int = 0): Double {
 if (isAvailable) { println("Peringatan: item '$title' tidak sedang dipinjam."); return 0.0 }
 isAvailable = true
 val fine = daysLate.coerceAtLeast(0) * calculateFinePerDay()
 println("Item '$title' berhasil dikembalikan.")
 if (fine > 0) println("Total denda: ${formatRupiah(fine)}")
 return fine
 }

 /** Menampilkan informasi umum item. */
 open fun displayInfo() {
 println("ID: $id | Judul: $title | Tahun: $year")
 println("Jenis: ${getItemType()} | Status: ${if (isAvailable) "Tersedia" else "Dipinjam"}")
 println("Denda/hari: ${formatRupiah(calculateFinePerDay())} | Maks. pinjam: ${getMaxBorrowDays()} hari")
 }
}

```



```
}
}
```

```
/** Memformat nominal menjadi rupiah tanpa angka desimal. */
fun formatRupiah(amount: Double): String = "Rp
%,d".format(amount.toLong()).replace(',', ' ')
```[cite: 4]
```

```
**2. Book.kt**
```

```
```kotlin
```

```
/** Buku yang tersedia di perpustakaan. */
```

```
class Book(
 id: String,
 title: String,
 year: Int,
 /** Penulis buku. */ val author: String,
 /** Jumlah halaman buku. */ val pages: Int,
 /** Genre buku. */ val genre: String
) : Item(id, title, year) {
 /** Mengembalikan denda keterlambatan buku. */
 override fun calculateFinePerDay() = 2000.0

 /** Mengembalikan kategori item ini. */
 override fun getItemType() = "Buku"

 /** Mengembalikan batas waktu peminjaman buku. */
 override fun getMaxBorrowDays() = 14

 /** Menampilkan informasi buku. */
 override fun displayInfo() {
 super.displayInfo()
 println("Penulis: $author | Halaman: $pages | Genre: $genre")
 }
}
```

```
```[cite: 2]
```

```
**3. Journal.kt**
```

```
```kotlin
```

```
/** Jurnal yang tersedia di perpustakaan. */
```

```
class Journal(
 id: String,
 title: String,
 year: Int,
 /** Penerbit jurnal. */ val publisher: String,
 /** Volume jurnal. */ val volume: Int,
 /** Nomor edisi jurnal. */ val issueNumber: Int
) : Item(id, title, year) {
 /** Mengembalikan denda keterlambatan jurnal. */
```



```
override fun calculateFinePerDay() = 3000.0
```

```
/** Mengembalikan kategori item ini. */
```

```
override fun getItemType() = "Jurnal"
```

```
/** Mengembalikan batas waktu peminjaman jurnal. */
```

```
override fun getMaxBorrowDays() = 7
```

```
/** Menampilkan informasi jurnal. */
```

```
override fun displayInfo() {
```

```
 super.displayInfo()
```

```
 println("Penerbit: $publisher | Volume: $volume | Edisi: $issueNumber")
```

```
}
```

```
}
```

```
```[cite: 5]
```

```
**4. DVD.kt**
```

```
```kotlin
```

```
/** DVD yang tersedia di perpustakaan. */
```

```
class DVD(
```

```
 id: String,
```

```
 title: String,
```

```
 year: Int,
```

```
 /** Sutradara film. */ val director: String,
```

```
 /** Durasi film dalam menit. */ val duration: Int,
```

```
 /** Genre DVD. */ val genre: String
```

```
) : Item(id, title, year) {
```

```
 /** Mengembalikan denda keterlambatan DVD. */
```

```
 override fun calculateFinePerDay() = 5000.0
```

```
/** Mengembalikan kategori item ini. */
```

```
override fun getItemType() = "DVD"
```

```
/** Mengembalikan batas waktu peminjaman DVD. */
```

```
override fun getMaxBorrowDays() = 3
```

```
/** Menampilkan informasi DVD. */
```

```
override fun displayInfo() {
```

```
 super.displayInfo()
```

```
 println("Sutradara: $director | Durasi: $duration menit | Genre:
```

```
$genre")
```

```
}
```

```
}
```

```
```[cite: 3]
```

```
**5. TransactionStatus.kt**
```

```
```kotlin
```

```
/** Merepresentasikan semua kemungkinan status dalam siklus transaksi. */
```



```
sealed class TransactionStatus {
 /** Menghasilkan label status yang mudah dibaca. */
 abstract fun display(): String

 /** Bernilai benar jika status ini tidak dapat diubah lagi. */
 fun isFinal(): Boolean = this is Returned || this is Cancelled

 /** Item sedang dipinjam. */
 object Borrowed : TransactionStatus() {
 override fun display() = "Dipinjam"
 }

 /** Item telah dikembalikan tepat waktu. */
 object Returned : TransactionStatus() {
 override fun display() = "Dikembalikan"
 }

 /** Item dikembalikan terlambat. */
 data class Overdue(
 /** Jumlah hari keterlambatan. */ val daysLate: Int
) : TransactionStatus() {
 override fun display() = "Terlambat ($daysLate hari)"
 }

 /** Transaksi dibatalkan. */
 object Cancelled : TransactionStatus() {
 override fun display() = "Dibatalkan"
 }
}

```[cite: 10]

**6. Transaction.kt**
```kotlin
import java.time.LocalDate

/** Transaksi peminjaman yang menghubungkan anggota dan item. */
class Transaction(
 /** Pengidentifikasi transaksi yang unik. */ val id: String,
 /** Item yang dipinjam. */ val item: Item,
 /** Anggota yang meminjam item. */ val member: Member,
 /** Tanggal saat item dipinjam. */ val borrowDate: String =
 LocalDate.now().toString(),
 /** Status transaksi saat ini. */ var status: TransactionStatus =
 TransactionStatus.Borrowed
) {
 /** Mengembalikan item dan mencatat status pengembaliannya. */
 fun returnItem(daysLate: Int): Double {
 if (status.isFinal()) {
```



```
 println("Transaksi $id sudah selesai dan tidak dapat diubah.")
 return 0.0
 }
 val lateDays = daysLate.coerceAtLeast(0)
 val fine = item.returnItem(lateDays)
 status = if (lateDays > 0) TransactionStatus.Overdue(lateDays) else
TransactionStatus.Returned
 return fine
}
```

```
 /** Membatalkan transaksi ini dan membuat item tersedia kembali. */
 fun cancel() {
 if (status.isFinal()) {
 println("Transaksi $id sudah selesai dan tidak dapat dibatalkan.")
 } else {
 status = TransactionStatus.Cancelled
 item.returnItem()
 }
 }
}
```

```
 /** Menampilkan detail transaksi. */
 fun displayTransaction() = println("$id | ${item.title} (${item.id}) |
${member.name} | $borrowDate | ${status.display()}")
}
```[cite: 9]
```

****7. Member.kt****

```kotlin

/\*\* Anggota perpustakaan yang sudah terdaftar. \*/

class Member{

/\*\* Pengidentifikasi anggota yang unik. \*/ val id: String,

/\*\* Nama anggota. \*/ val name: String,

private val email: String,

private val phone: String

) {

private val transactions = mutableListOf<Transaction>()

/\*\* Jumlah seluruh transaksi yang dilakukan anggota ini. \*/

val transactionCount: Int get() = transactions.size

/\*\* Total denda dari transaksi yang terlambat. \*/

val totalFines: Double get() = transactions.sumOf { transaction ->

(transaction.status as? TransactionStatus.Overdue)?.let { it.daysLate \*
transaction.item.calculateFinePerDay() } ?: 0.0

}

/\*\* Jumlah item yang sedang dipinjam. \*/

val activeBorrows: Int get() = transactions.count { it.status is
TransactionStatus.Borrowed }



```
/** Mengembalikan alamat email anggota. */
fun getEmail(): String = email

/** Mengembalikan nomor telepon anggota. */
fun getPhone(): String = phone

/** Meminjam item yang tersedia jika anggota memiliki kurang dari tiga
pinjaman aktif. */
fun borrowItem(item: Item): Transaction? {
 if (!item.isAvailable) { println("Item ${item.id} tidak tersedia.");
return null }
 if (activeBorrows >= 3) { println("$name sudah mencapai batas 3
peminjaman aktif."); return null }
 if (!item.borrow()) return null
 val transaction = Transaction("TRX-${System.currentTimeMillis()}",
item, this)
 transactions.add(transaction)
 println("$name berhasil meminjam '${item.title}'.")
 return transaction
}

/** Mengembalikan pinjaman aktif anggota untuk suatu item. */
fun returnItem(item: Item, daysLate: Int = 0): Double {
 val transaction = transactions.firstOrNull { it.item === item &&
it.status is TransactionStatus.Borrowed }
 if (transaction == null) { println("Tidak ada transaksi aktif untuk
item ${item.id}."); return 0.0 }
 return transaction.returnItem(daysLate)
}

/** Mengembalikan salinan transaksi anggota yang tidak dapat diubah. */
fun getTransactions(): List<Transaction> = transactions.toList()

/** Menampilkan informasi anggota. */
fun displayInfo() = println("ID: $id | Nama: $name | Email: $email |
Telepon: $phone | Total Pinjam: $transactionCount | Aktif: $activeBorrows |
Denda: ${formatRupiah(totalFines)}")

/** Menampilkan riwayat transaksi anggota. */
fun displayTransactions() {
 println("Riwayat transaksi $name:")
 if (transactions.isEmpty()) println("Belum ada transaksi.") else
transactions.forEach { it.displayTransaction() }
}

}

```[cite: 8]
```



```
```kotlin
/** Mengelola item, anggota, dan transaksi perpustakaan. */
class Library(
 /** Nama perpustakaan ini. */ val name: String
) {
 private val items = mutableListOf<Item>()
 private val members = mutableListOf<Member>()
 private val transactions = mutableListOf<Transaction>()

 /** Jumlah seluruh item yang terdaftar. */
 val totalItems: Int get() = items.size

 /** Jumlah item yang tersedia. */
 val availableItems: Int get() = items.count { it.isAvailable }

 /** Jumlah seluruh anggota yang terdaftar. */
 val totalMembers: Int get() = members.size

 /** Jumlah seluruh transaksi peminjaman. */
 val totalTransactions: Int get() = transactions.size

 /** Menambahkan item ke koleksi. */
 fun addItem(item: Item) {
 items.add(item)
 println("Item ${item.id} ditambahkan.")
 }

 /** Menambahkan beberapa item ke koleksi. */
 fun addItems(vararg newItems: Item) = newItems.forEach(::addItem)

 /** Mencari item berdasarkan ID. */
 fun findItem(id: String): Item? = items.find { it.id.equals(id, true) }

 /** Mencari item berdasarkan kata kunci judul atau ID. */
 fun searchItems(keyword: String): List<Item> = items.filter {
 it.id.contains(keyword, true) || it.title.contains(keyword, true)
 }

 /** Mendaftarkan anggota jika ID-nya belum digunakan. */
 fun registerMember(id: String, name: String, email: String, phone: String):
 Boolean {
 if (findMember(id) != null) {
 println("ID anggota $id sudah digunakan.")
 return false
 }
 members.add(Member(id, name, email, phone))
 println("Anggota $name berhasil didaftarkan.")
 return true
 }
}
```



```
}

/** Mencari anggota berdasarkan ID. */
fun findMember(id: String): Member? = members.find { it.id.equals(id, true)
}

/** Membuat transaksi peminjaman untuk anggota dan item. */
fun borrowItem(memberId: String, itemId: String): Transaction? {
 val member = findMember(memberId) ?: run { println("Anggota $memberId
tidak ditemukan."); return null }
 val item = findItem(itemId) ?: run { println("Item $itemId tidak
ditemukan."); return null }
 return member.borrowItem(item)?.also { transactions.add(it) }
}

/** Mengembalikan item atas nama anggota. */
fun returnItem(memberId: String, itemId: String, daysLate: Int = 0): Double
{
 val member = findMember(memberId) ?: run { println("Anggota $memberId
tidak ditemukan."); return 0.0 }
 val item = findItem(itemId) ?: run { println("Item $itemId tidak
ditemukan."); return 0.0 }
 return member.returnItem(item, daysLate)
}

/** Menampilkan informasi lengkap semua item. */
fun displayAllItems() {
 println("\n=== SEMUA ITEM ($totalItems, tersedia $availableItems) ===")
 items.forEach {
 it.displayInfo()
 println()
 }
}

/** Menampilkan daftar singkat item yang tersedia. */
fun displayAvailableItems() {
 println("\n=== ITEM TERSEDIA ===")
 items.filter { it.isAvailable }.forEach {
 println("${it.id} - ${it.title} (${it.getItemType()})")
 }
}

/** Menampilkan informasi semua anggota. */
fun displayAllMembers() {
 println("\n=== SEMUA ANGGOTA ===")
 members.forEach { it.displayInfo() }
}

/** Menampilkan semua transaksi perpustakaan. */
```



```
fun displayAllTransactions() {
 println("\n== SEMUA TRANSAKSI ==")
 transactions.forEach { it.displayTransaction() }
}

/** Menampilkan laporan ringkasan perpustakaan ini. */
fun displayReport() {
 val totalFines = members.sumOf { it.totalFines }
 println("\n== LAPORAN $name ==\nTotal item: $totalItems\nTersedia:
$availableItems\nDipinjam: ${totalItems - availableItems}\nTotal anggota:
$totalMembers\nTotal transaksi: $totalTransactions\nTotal denda:
${formatRupiah(totalFines)}")
}

```[cite: 6]

**9. Main.kt (Library)**
```kotlin
/** Menjalankan contoh penggunaan Sistem Manajemen Perpustakaan Digital. */
fun main() {
 val library = Library("Perpustakaan Kampus")
 val book1 = Book("B001", "Pemrograman Kotlin", 2023, "Budi Santoso", 350,
"Programming")
 val book2 = Book("B002", "Dasar-Dasar OOP", 2022, "Siti Rahayu", 280,
"Education")
 val journal1 = Journal("J001", "Jurnal Teknologi Informasi", 2023, "ITB",
15, 2)
 val journal2 = Journal("J002", "Jurnal Pendidikan", 2022, "UGM", 10, 1)
 val dvd1 = DVD("D001", "Inception", 2010, "Christopher Nolan", 148, "Sci-
Fi")
 val dvd2 = DVD("D002", "The Matrix", 1999, "Wachowski", 136, "Action")

 println("=== INISIALISASI ${library.name} ===")
 library.addItem(book1, book2, journal1, journal2, dvd1, dvd2)
 library.registerMember("M001", "Ahmad Fauzi", "ahmad@email.com",
"08123456789")
 library.registerMember("M002", "Dewi Lestari", "dewi@email.com",
"08129876543")
 library.registerMember("M003", "Rizky Pratama", "rizky@email.com",
"08125678901")
 library.displayAllItems()

 println("=== PEMINJAMAN ===")
 library.borrowItem("M001", "B001"); library.borrowItem("M001", "D001")
 library.borrowItem("M002", "J001"); library.borrowItem("M003", "B002")
 library.displayAvailableItems()
 library.findMember("M001")?.displayTransactions();
 library.findMember("M002")?.displayTransactions()
}
```



```
println("\n=== PENGEMBALIAN ===")
library.returnItem("M001", "B001"); library.returnItem("M002", "J001", 3)
library.findMember("M001")?.displayTransactions();
library.findMember("M002")?.displayTransactions()

println("\n=== POLIMORFISME ===")
val polymorphicItems: List<Item> = listOf(book1, journal2, dvd2)
polymorphicItems.forEach { println("${it.getItemType()} - Denda/hari:
${formatRupiah(it.calculateFinePerDay())}") }

println("\n=== SEALED CLASS ===")
val statuses: List<TransactionStatus> = listOf(TransactionStatus.Borrowed,
TransactionStatus.Returned, TransactionStatus.Overdue(5),
TransactionStatus.Cancelled)
statuses.forEach { status -> when (status) {
 TransactionStatus.Borrowed -> println(status.display())
 TransactionStatus.Returned -> println(status.display())
 is TransactionStatus.Overdue -> println(status.display())
 TransactionStatus.Cancelled -> println(status.display())
} }

println("\n=== SMART CASTING ===")
val selectedItem: Item = book1
when (selectedItem) { is Book -> println("Item ${selectedItem.id} adalah
Buku"); is Journal -> println("Item ${selectedItem.id} adalah Jurnal"); is DVD
-> println("Item ${selectedItem.id} adalah DVD") }
println("Casting B001 ke DVD: ${selectedItem as? DVD ?: "null (bukan
DVD)"}")
println("\n=== ENKAPSULASI ===\n'isAvailable' tidak dapat diubah langsung
karena private set. Email anggota juga private dan diakses melalui
getEmail().")
library.displayReport()
}

```[cite: 7]
```